

Design an Embedded C simulation for a healthcare monitoring system with interrupt sampling and fail-safe mechanisms.

Program:

```
/* Embedded C Simulation: Healthcare Monitoring System */

#include <stdio.h> #include <stdbool.h>

/* Sensor limits */ #define HR_MIN 40 #define HR_MAX 120

#define TEMP_MIN 35.0 #define TEMP_MAX 38.5

#define SPO2_MIN 90

/* System states */ typedef enum { NORMAL, WARNING, CRITICAL, FAIL_SAFE }
SystemState;

/* Sensor data structure */ typedef struct { int heartRate; float temperature; int spo2; }
SensorData;

/* Global variables */ volatile bool sampleFlag = false; volatile bool sensorFault = false;

SensorData patientData; SystemState systemState = NORMAL;

/----- Timer Interrupt Service Routine -----
-----/

void TIMER_ISR(void) { /* Timer interrupt occurs periodically. Example: every 1 second. */

sampleFlag = true;

}

/----- Sensor Reading Functions -----
-----/

int ReadHeartRate(void) { /* Simulation of heart-rate sensor */ return 75; }

float ReadTemperature(void) { /* Simulation of temperature sensor */ return 36.8; }
```

```

int ReadSpO2(void) { /* Simulation of SpO2 sensor */ return 97; }

/----- Sensor Sampling -----
-----/

void SampleSensors(void) { patientData.heartRate = ReadHeartRate();
patientData.temperature = ReadTemperature(); patientData.spo2 = ReadSpO2(); }

/----- Sensor Validation -----
-----/

bool ValidateSensors(void) { /* Check for physically impossible or invalid sensor readings.
*/

if (patientData.heartRate <= 0)
    return false;

if (patientData.temperature < 20.0 ||
    patientData.temperature > 50.0)
    return false;

if (patientData.spo2 <= 0 ||
    patientData.spo2 > 100)
    return false;

return true;

}

/----- Patient Condition Monitoring -----
-----/

void AnalyzePatientCondition(void) { bool critical = false; bool warning = false;

/* Heart-rate analysis */
if (patientData.heartRate < HR_MIN ||
    patientData.heartRate > HR_MAX)
{
    critical = true;
}

```

```

/* Temperature analysis */
if (patientData.temperature < TEMP_MIN ||
    patientData.temperature > TEMP_MAX)
{
    warning = true;
}

/* SpO2 analysis */
if (patientData.spo2 < SPO2_MIN)
{
    critical = true;
}

/* Determine system state */
if (critical)
{
    systemState = CRITICAL;
}
else if (warning)
{
    systemState = WARNING;
}
else
{
    systemState = NORMAL;
}

}

/----- Fail-Safe Mechanism -----
-----/

void EnterFailSafe(void) { systemState = FAIL_SAFE;

/*
    Disable unsafe operations.
    Activate emergency indication.
*/

```

```
printf("FAIL-SAFE MODE ACTIVATED\n");
printf("Emergency alarm ON\n");
```

```
/*
```

```
    In real embedded hardware:
```

- Activate buzzer
- Turn on warning LED
- Notify medical staff
- Switch to backup sensor/system
- Place actuators in a safe state

```
*/
```

```
}
```

```
/----- Alarm Handling -----  
-----/
```

```
void HandleAlarm(void) { switch (systemState) { case NORMAL: printf("Status:  
NORMAL\n"); break;
```

```
    case WARNING:
```

```
        printf("WARNING: Abnormal reading detected\n");  
        break;
```

```
    case CRITICAL:
```

```
        printf("CRITICAL: Immediate attention required\n");  
        printf("Emergency alarm ON\n");  
        break;
```

```
    case FAIL_SAFE:
```

```
        printf("System in FAIL-SAFE state\n");  
        break;
```

```
}
```

```
}
```

```
/----- Display Patient Data -----  
-----/
```

```
void DisplayData(void) { printf("\nPatient Data\n"); printf("Heart Rate : %d bpm\n",  
patientData.heartRate);
```

```
printf("Temperature: %.2f C\n",  
        patientData.temperature);
```

```
printf("SpO2      : %d %%\n",  
        patientData.spo2);
```

```
}
```

```
/----- Main Program -----  
-----/
```

```
int main(void) { printf("Healthcare Monitoring System Started\n");
```

```
/* Hardware initialization */  
/* Timer_Init(); */  
/* Sensor_Init(); */  
/* Alarm_Init(); */
```

```
while (1)  
{
```

```
    /*  
        Main loop waits for timer interrupt.  
    */
```

```
    if (sampleFlag == true)  
    {  
        /* Clear flag */  
        sampleFlag = false;
```

```
        /* Read sensors */  
        SampleSensors();
```

```
        /* Validate sensor readings */
```

```

    if (!ValidateSensors())
    {
        sensorFault = true;
    }

    /* Check for sensor/system fault */
    if (sensorFault)
    {
        EnterFailSafe();
        continue;
    }

    /* Analyze patient condition */
    AnalyzePatientCondition();

    /* Display data */
    DisplayData();

    /* Handle alarms */
    HandleAlarm();
}

/*
CPU can enter low-power/sleep mode here
until the next interrupt.
*/

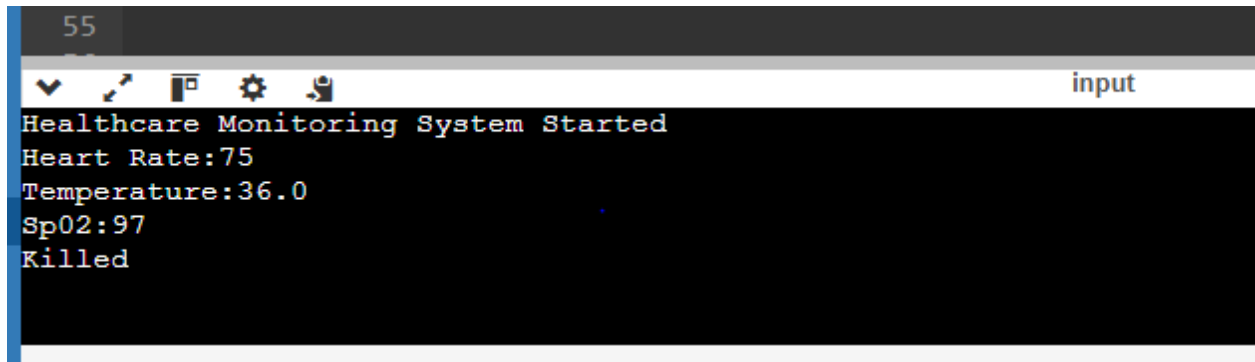
/* SleepUntilInterrupt(); */
}

return 0;

}

```

Output:



```
55
input
Healthcare Monitoring System Started
Heart Rate:75
Temperature:36.0
SpO2:97
Killed
```